

Activity Agent 设计文档

v0.2.0 | SOP-v2 架构 (8-phase · 12-tool · single-confirm) | 核心文件 : plan-state.ts / activity-tools.ts / tool-wrapper.ts

一、Planning 策略

核心理念：单次确认 + 单次追问。 用户仅需在最终方案时确认一次，其余环节 LLM 自主完成。

```
[用户输入] → intent_capture → [clarifying×1] → planning(自动) → plan_confirm ☆ → executing → completed
```

八阶段状态机由 PHASE_TRANSITIONS DAG 硬约束，PlanStateManager.transition() 校验每次跳转合法性。

三个硬约束：

- **Single-Confirm** — plan_confirm 是唯一确认点。planning 阶段 LLM 自主完成 weather/POI/route/opening-hours 全部调用，不向用户逐项确认。
- **1-Clarify Limit** — MAX_CLARIFICATIONS=1，第 2 次调用 ask_clarification 返回 MAX_CLARIFICATIONS_EXCEEDED，强制用 fallbackDefaults 推进。
- **Critical Fields Auto-Fill** — 5 个关键字段 (date/startTime/partySize/departurePoint/budgetPerPerson) 缺失时，intent_parse 自动从 user-preferences.ts 跨 session 记忆 (≥50% 出现率阈值) 补全，齐全后自动跳转 planning。

用户确认分类 (classifyUserConfirmation) : confirm (确认/好的/ok) → executing ; modify (修改/调整) ↔ planning ; reject/ambiguous → intent_capture。

二、工具调用链路

全景架构： Browser → POST /api/agent/[id] → startRpcSession() → createAgentSession({ customTools: 12 }) → AgentSessionWrapper.send("prompt") → advancePlanPhase(msg) → session.prompt() → LLM 推理 → tool.execute() [guard → retry → fallback]。前端通过 SSE /api/agent/[id]/events 接收 tool_execution_start/end + message_end 事件，同时 GET /api/plan-state/[id] 每 1.5s 轮询。

12 工具 Phase 白名单：

Phase	工具	约束
意图	intent_parse / ask_clarification	clarify 硬限 1 次
规划	get_weather / search_activities / search_restaurants / check_opening_hours / compute_route	LLM 自主调用，无用户交互
执行	reservation_exec / query_booking / retry_booking	reservation_exec 仅 executing
持久化	plan_save / plan_load	executing+ / idle+

Phase Guard 三层防线：

1. TOOL_PHASE_RULES — 工具注册前静态白名单
2. guardToolCallWithActive() — wrapper beforeExecute 读取全局 phase，不匹配返回 PHASE_GUARD JSON (不抛异常)
3. 工具内部自校验 — intent_parse(submitPlan=true) 仅在 planning 合法，plan_confirm/executing 阶段调用返回 SUBMIT_PLAN_OUT_OF_PHASE

Tool Wrapper 执行链（lib/tool-wrapper.ts，所有 12 工具统一包装）：

```
beforeExecute(phase guard)
  → execute + Promise.race(timeout)
  → [失败] → retry(exponential backoff)
  → [耗尽] → fallback handler
    → [也失败] → 结构化 JSON 错误（永不抛异常）
    → 记录 ToolMetrics
```

三类预设策略：

类别	重试	退避	超时	Fallback
查询（POI/weather/route/hours）	2 次	exponential 100ms → 1s	5s	空结果 + "用 LLM 知识继续"
写操作（reservation/retry）	3 次	exponential 300ms → 3s	8s	"预订暂不可用，稍后重试"
持久化/意图	1 次	fixed 100ms	3s	无（错误是确定的）

三、异常处理机制

核心原则：所有错误以结构化 JSON 返回，永不抛异常到 AgentSession 层。LLM 总能读到错误信息，用户体验不中断。

五层防线：

层	机制	失败行为
L1 Phase Guard	beforeExecute 校验 tool-phase	PHASE_GUARD JSON → LLM 调整行为
L2 Input Validation	Booking: 日期/时间/人数/状态；Route: 起终点必填	BookingError(code) 或 error JSON
L3 Timeout	Promise.race（查询 5s / 写 8s / 持久 3s）	超时 → 进入 L4
L4 Retry	exponential backoff: base × 2^attempt	耗尽 → 进入 L5
L5 Fallback	查询 → 空结果+LLM 提示；写 → "暂不可用"	fallback 失败 → TOOL_EXECUTION_FAILED JSON

BookingError 体系（8 种错误码）：RESTAURANT_NOT_FOUND / INVALID_DATE / PAST_DATE / INVALID_PARTY_SIZE / INVALID_STATE / ORDER_NOT_FOUND 等。suggestFix() 为每种提供中文修复指引。

预订状态机容错：pending → processing → confirmed → notified；异常分支 failed → retry_booking → re-processing。异步模拟外部 API（800ms 延迟，10% 失败率），订单持久化到 ~/.pi/agent/bookings/，重启可恢复。

全局健壮性：空白输入前端拦截；意图不明走 clarify → default 降级；POI 不存在 / 营业时间不匹配 → 结构化 error → LLM 自主换 POI；持久化失败 → catch+log 不阻塞；10min 无活动 → idle timer 自动 destroy。

可观测性：ToolMetrics 环形缓冲（200 条）记录每次调用的 attempts/duration/success/fallback；SSE 实时推送；p0-smoke-test.ts（126 asserts）+ e2e-real-llm-test.ts（24 asserts）覆盖完整链路。